



**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ИНФОРМАЦИОННЫЕ СИСТЕМЫ В ХТ

Лекция 24. Методология DevOps

Содержание лекции

В данной лекции будут рассмотрены следующие темы:

-  Постановка задачи быстрой поставки приложений
-  Определение DevOps
-  Практики DevOps
-  Ключевые инструменты DevOps

Поставка приложений

- 📡 Оперативный выпуск новых продуктов/версий – один из основных приоритетов компании
 - 🕒 Возможность быстрее учитывать обратную связь
 - 🕒 Вводить в эксплуатацию новые функции в максимально короткие сроки
- 📡 В большей степени актуально для компаний, процессы которых непосредственно связаны с их IT технологиями
- 📡 Короткий time to market (TTM) – одно из ключевых преимуществ компании в конкурентной среде

Поставка приложений – сложности

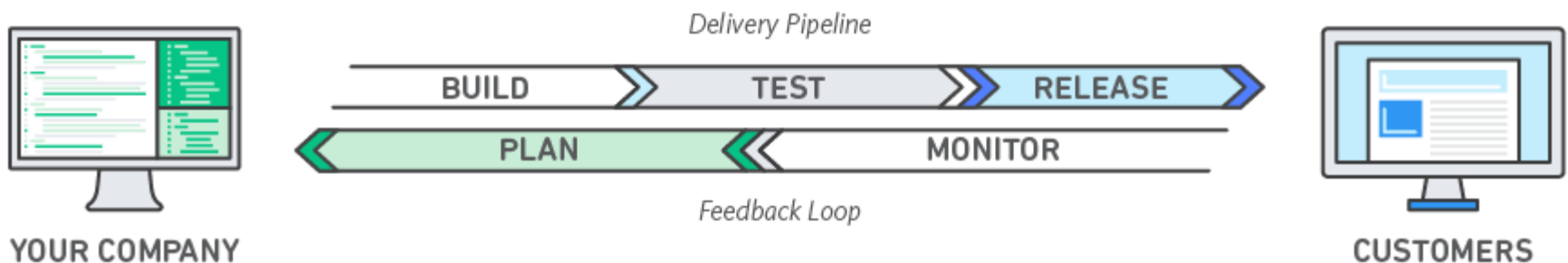
- ☎ Текущие практики, принятые в некоторых компаниях, которые зачастую препятствуют короткому ТТМ:
 - ☉ Классический «проектный» подход к разработке
 - ☉ Организационная структура компаний: отдел разработки, тестирования, поддержки, администрирования
 - ☉ Изначальная ориентация на редкие крупные релизы
 - ☉ Отсутствие автоматизации
 - ☉ «Разобранные» проекты

DevOps

- DevOps - комбинация практик, подходов и инструментов, которые увеличивают скорость поставки приложений и сервисов. Таким образом, выпуск следующих версий продуктов происходит чаще, чем в случае использования классических подходов к разработке и управлению инфраструктурой. Эта скорость позволяет лучше обеспечивать потребности клиентов и повышать конкурентоспособность создаваемых продуктов.

(с)

Amazon



DevOps – как это работает

- 📡 DevOps предполагает формирование самодостаточных проектных команд:
 - 📡 Разработчики
 - 📡 Администраторы
 - 📡 QA
 - 📡 Аналитики
 - 📡 Специалисты по безопасности
- 📡 Такие команды могут сами отвечать за обеспечение всех этапов жизненного цикла: от сбора требований до сопровождения разработанного ПО
- 📡 Инструменты автоматизации используются в самых дорогих и сложных процессах жизненного цикла: сборка, тестирование, развертывание и т.д.

DevOps – преимущества

-  Скорость бизнес-инноваций
-  Быстрое создание поставок
-  Надежность
-  Лучшие возможности масштабирования
-  Хорошая коммуникация между участниками команды

DevOps – аналогия с промышленным миром

- 🌐 DevOps – аналог промышленной автоматизации XX века для мира IT:
 - ⦿ Переход от ручного труда к машинному
 - ⦿ Производительность вырастает в разы
 - ⦿ Качество и масштабируемость на высоком уровне
 - ⦿ Появляется необходимость в новых компетенциях: организация конвейера, поддержание его бесперебойной работы

DevOps – ключевые практики

-  Непрерывная интеграция/доставка
-  Использование микросервисов
-  Системы управления конфигурацией
 -  Infrastructure as Code
-  Мониторинг и журналирование
-  Организация проектных команд

Непрерывная интеграция

-  Непрерывная интеграция – это практика разработки программного обеспечения, которая предполагает максимально интенсивную интеграцию работы, которая выполняется разными сотрудниками (разработчиками).
-  Каждый разработчик должен интегрировать свою работу в общий проект, как минимум, ежедневно.
-  Каждая интеграция должна проверяться на предмет возможных ошибок сборки/компиляции и на предмет успешности выполнения автоматических тестов.

Микросервисы

- ➊ Подход, предполагающий разделение функционала приложения на небольшие части – микросервисы.
- ➋ Каждый из них представляет атомарный функционал и интерфейс для работы с ним.
- ➌ Каждый микросервис может разрабатываться и разворачиваться независимо или группами.
- ➍ Появляется возможность более оперативно вносить точечные изменения в приложение на уровне отдельных сервисов

Системы управления конфигурацией

-  Отказ от внесения ручных изменений в конфигурацию сред развертывания
-  Полная автоматизация этого процесса для обеспечения надежности, повторяемости и предсказуемости этих процессов
-  Работа с конфигурацией ведется как с кодом, с применением соответствующих современных практик

Мониторинг и журналирование

-  Архитектура микросервисов создает определенные сложности, связанные с локализацией и устранением проблем, а также настройкой производительности
-  Мониторинг и журналирование необходимы для получения максимально быстрой обратной связи от сервисов
-  Мониторинг перестает быть прерогативой администраторов, разработчик должен принимать активное участие в процессе его настройки

Организация проектных команд

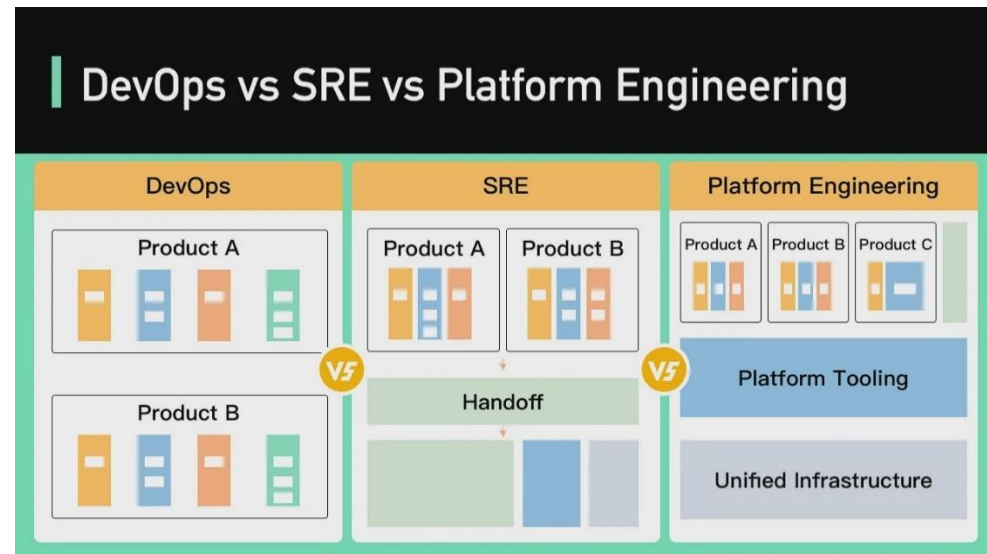
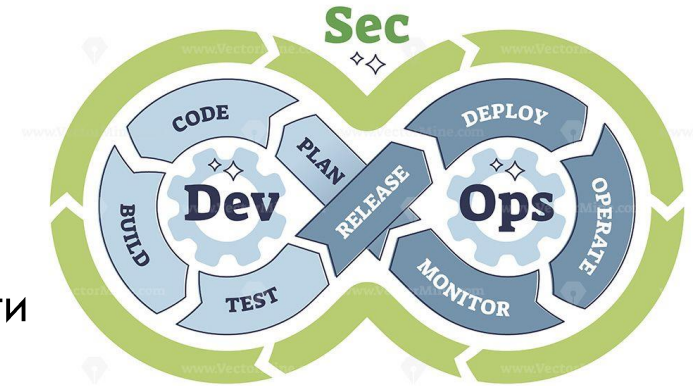
-  Проектная команда включает всех необходимых специалистов для разработки/поддержки систем
-  Администраторы погружены в специфику разрабатываемых систем и находятся на постоянной связи с разработчиками, предоставляя им инфраструктурные сервисы
-  Разработчик занимается не только приложением, но и тестами, а также конфигурацией

Инструменты непрерывной интеграции

-  Системы контроля версий
 -  Git + GitHub/GitLab
-  Системы непрерывной интеграции/доставки
 -  UrbanCode, Jenkins, TeamCity, Bamboo
-  Системы управления конфигурацией
 -  Ansible, Chef, SaltStack, Puppet
-  Системы контейнерной виртуализации (микросервисы)
 -  Docker + Kubernetes
-  Репозитории бинарных артефактов
 -  Artifactory, Nexus
-  Сборщики ПО
 -  Maven, Ant, Gradle

Тенденции в DevOps

- GitOps
- ChatOps
- DevSecOps
 - «Сдвиг в лево» решения задач безопасности
- Site Reliability Engineering (SRE)
 - Фокус на обеспечении доступности
- Platform Engineering
 - Internal Development Platform



Итоги

В данной лекции были рассмотрены следующие темы:

-  Постановка задачи быстрой поставки приложений
-  Определение DevOps
-  Практики DevOps
-  Ключевые инструменты DevOps